МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное
учреждение высшего образования
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ

Д. О. Шевяков, Д. Д. Савельева

ПРИКЛАДНЫЕ СРЕДСТВА ИНТЕРНЕТА ВЕЩЕЙ

Учебно-методическое пособие

ГУАП

Санкт-Петербург
2025

УДК
ББК
В

Рецензенты:
доктор технических наук, доцент *С. В. Мичурин*;
доктор технических наук, доцент *С. В. Беззатеев*

Утверждено
редакционно-издательским советом университета
в качестве учебно-методического пособия

Протокол №

Шевяков, Д. О.
В Прикладные средства интернета вещей: учеб.-метод. пособие / Д. О. Шевяков, Д. Д. Савельева. – СПб.: ГУАП, 2025. – ??? с.

Рассмотрены такие вопросы как: концептуальные основы интернета вещей, исторический контекст технологий интернета вещей, практическое применение технологии интернета вещей, и возможности интеграции технологии интернета вещей с другими современными инструментами. Содержится описание и порядок выполнения лабораторных работ и задания к ним.

Предназначено для студентов, обучающихся по направлениям 09.03.02 «Информационные системы и технологии», 11.03.02 «Инфокоммуникационные технологии и системы связи», 23.03.01 «Технология транспортных процессов», 27.03.03 «Системный анализ и управление», 24.03.02 «Системы управления движением и навигация», 25.03.01 «Техническая эксплуатация летательных аппаратов и двигателей» и может быть рекомендован к использованию при изучении дисциплин «Интернет вещей» и «ИТ-модуль Интернет вещей».

УДК
ББК

© Санкт-Петербургский государственный университет аэрокосмического приборостроения, 2025

1. КОНЦЕПТУАЛЬНЫЕ ОСНОВЫ ИНТЕРНЕТА ВЕЩЕЙ

1.1. Сущность и базовое определение

Современный этап развития цифровых технологий характеризуется переходом от антропоцентрической модели взаимодействия с информационными системами к распределенной экосистеме, где ключевыми факторами выступают объекты физического мира. Эта парадигма получила название «Интернет вещей» (*Internet of Things, IoT*). Концепция *IoT* представляет собой новую фазу цифровой революции, суть которой заключается в наделении обычных материальных объектов способностью к сбору, обработке, передаче данных и автономному взаимодействию друг с другом посредством глобальной сети, минимизируя или полностью исключая необходимость человеческого посредничества [1].

В наиболее общем виде, Интернет вещей – это совокупность устройств, связанных между собой и способных принимать и передавать информацию по беспроводным сетям без вмешательства человека. Однако в научной и профессиональной литературе существует множество более строгих и детализированных определений [2].

Ключевым аспектом *IoT* является не просто подключение устройства к сети, а создание целостной киберфизической системы, в которой данные от физических объектов («вещей») используются для анализа, принятия решений и обратного воздействия на физический мир. «Вещью» в контексте IoT может выступать любой объект, оснащенный сенсорами, исполнительными механизмами и сетевым интерфейсом: от датчика температуры в цеху и системы навигации в автомобиле до имплантируемого медицинского устройства [3].

1.2. Ключевые характеристики *IoT*-систем

Любая полноценная *IoT*-система обладает рядом характерных признаков:

1. Связность: Способность устройств подключаться к локальным или глобальным сетям (*LPWAN, Wi-Fi, Bluetooth, 5G*, спутниковые сети).

2. Сенсорика: Наличие датчиков для сбора данных об окружающей среде (температура, давление, движение, местоположение и т. д.).

3. Взаимодействие: Способность устройств не только получать, но и передавать команды, воздействуя на физические процессы.

4. Автономность: Минимизация роли человека в рутинных операциях; устройства способны самостоятельно выполнять задачи на основе заложенных алгоритмов.

5. Интеллектуальность: Использование методов анализа данных, машинного обучения и искусственного интеллекта для выявления паттернов, прогнозирования и оптимизации процессов.

Разные организации и исследователи предлагают свои критерии для идентификации феномена *IoT*. Одним из наиболее цитируемых является определение, предложенное консалтинговым подразделением *Cisco IBSG (Internet Business Solutions Group)*. Согласно ему, Интернет вещей – это момент времени, когда количество «вещей» или материальных объектов, подключенных к Интернету, превысило число людей, пользующихся Всемирной паутиной [4].

Это количественное определение вводит четкий, измеримый критерий для определения точки отсчета существования *IoT* как массового явления, согласно этому критерию, переломный момент наступил между 2008 и 2010 годами.

Альтернативные подходы, например, от организаций вроде *IEEE* или *ITU*, делают акцент на архитектурных и функциональных моделях, описывая *IoT* как глобальную инфраструктуру, которая обеспечивает продвинутые сервисы за счет взаимосвязи физических и виртуальных вещей на основе существующих и развивающихся интернет-технологий [5].

Таким образом, концепция *IoT* является многогранной, объединяя в себе технологический, экономический и социальный аспекты. Она знаменует собой переход от Интернета как средства коммуникации между людьми к Интернету как платформе для коммуникации между машинами (*M2M – Machine-to-Machine*) и интеграции цифрового и физического миров.

2. ИСТОРИЧЕСКАЯ ЭВОЛЮЦИЯ КОНЦЕПЦИИ ИНТЕРНЕТА ВЕЩЕЙ

2.1. Ранние предпосылки и пророческие идеи (1920-е – 1980-е гг.)

Истоки концепции лежат в первой половине *XX* века, одним из первых провидцев был Никола Тесла. В 1926 году в интервью журналу «*Collier's*» он заявил: «Когда беспроводная связь будет применена идеально, вся земля превратится в один большой мозг, что, в сущности, и есть, причем все вещи станут частью единого целого, и мы сможем общаться друг с другом мгновенно, независимо от расстояния. Не только это, но и с помощью телевидения и телефонии мы будем видеть и слышать друг друга так же прекрасно, как если бы мы находились лицом к лицу, несмотря на разделяющие нас тысячи миль; и инструменты, благодаря которым мы сможем это делать, будут удивительно простыми по сравнению с нашими нынешними телефонами. Человек сможет носить один такой в жилетном кармане».

Это высказывание с удивительной точностью предвосхитило не только идею глобальной связанности устройств, но и появление компактных мобильных устройств (смартфонов), являющихся сегодня ключевым элементом многих *IoT*-экосистем.

Следующим значимым шагом стало создание в 1969 году сети *ARPANET* – прародителя современного Интернета. Хотя изначально она связывала компьютеры, была заложена базовая сетевая архитектура, без которой *IoT* был бы невозможен.

В 1980-90-х годах появляются первые практические примеры подключения обычных устройств к сети. Символическим «первым устройством» Интернета вещей часто называют модернизированный кулер для напитков в университете Карнеги-Меллона. В начале 1980-х годов местные программисты подключили к сети датчики, чтобы удаленно проверять, есть ли в аппарате охлажденные банки с газировкой, экономя время на поход к пустому автомату. Это был примитивный, но наглядный пример принципа *IoT*: физический объект передает информацию о своем состоянии в сеть без прямого участия человека[6].

Термин «Интернет вещей» был предложен независимо двумя исследователями. В 1999 году Кевин Эштон, сооснователь *Auto-ID Center* при *MIT*, использовал его в презентации для *Procter*

& Gamble, описывая систему радиочастотной идентификации (*RFID*) для управления цепочками поставок. Именно его формулировка получила широкое распространение. Почти одновременно, в том же году, Билл Джой в своей речи на Всемирном экономическом форуме в Давосе предложил схожую концепцию «*Device-to-Device» (D2D)* коммуникации как часть своей модели «Шести Вебов».

2.2. Технологическая конвергенция и точка перелома (1990-е – 2000-е гг.)

Для реализации концепции *IoT* потребовалась конвергенция нескольких ключевых технологий:

– Миниатюризация и удешевление процессоров и сенсоров.

– Развитие беспроводных сетей связи (от *GSM* к *3G*, *Wi-Fi, Bluetooth*).

– Стандартизация IP-протоколов, что позволило унифицировать общение между разнородными устройствами.

– Появление облачных вычислений, предоставивших практически неограниченные ресурсы для хранения и обработки данных с миллионов устройств [5].

Как указывает анализ *Cisco IBSG*, в 2003 году на 6,3 млрд человек населения Земли приходилось лишь 500 млн подключенных устройств [4]. Это соотношение – 0,08 устройства на человека – было явно недостаточным для зарождения глобальной экосистемы. Технологии были дорогими, а сценарии применения – ограниченными.

Переломный момент, согласно критерию *Cisco*, наступил в период 2008-2009 годов. Катализатором этого процесса стало стремительное распространение смартфонов. Анонс Стивом Джобсом *iPhone* в 2007 году стал переломным моментом, изменившим рынок мобильных устройств. Смартфон, будучи универсальным устройством с множеством датчиков, мощным процессором и постоянным подключением к интернету, стал не только потребителем, но и хабом для других *IoT*-устройств [7].

К 2010 году, как приведено в исходных данных, количество подключенных устройств достигло 12,5 млрд при населении Земли в 6,8 млрд человек. Соотношение составило 1,84 устройства на душу населения. Этот количественный сдвиг ознаменовал официальный переход от «Интернета людей» к «Интернету вещей». Устройств, общающихся друг с другом, стало больше, чем людей, использующих сеть для коммуникации [4].

2.3. Современный этап и перспективы (2010-е гг. – по настоящее время)

С 2010 года *IoT* вступил в фазу взрывного роста. Технология перестала быть экзотикой и стала общепринятой практикой. Сформировались ключевые сегменты рынка:

1. Промышленный *IoT* (*IIoT*): «Умные» фабрики, предиктивная аналитика оборудования, цифровые двойники.

2. Умный дом: Системы безопасности, климат-контроля, управления бытовой техникой.

3. Умный город: Интеллектуальные системы освещения, управления трафиком, утилизации отходов.

4. Телемедицина и носимые устройства: Фитнес-трекеры, умные часы, удаленный мониторинг пациентов.

5. Точное земледелие: Мониторинг состояния почвы, автоматизированный полив, управление сельхозтехникой [3,8].

Экономические показатели подтверждают этот рост. Как отмечают аналитики *IDC*, в 2023 году глобальные расходы на *IoT*-технологии достигли 805,7 млрд долларов США, демонстрируя рост на 10,6% по сравнению с 2022 годом [9]. Прогнозируется, что к 2026 году этот рынок превысит триллион долларов, а количество подключенных устройств к концу 2025 года составит 55,7 млрд.

Современный этап характеризуется интеграцией *IoT* с другими сквозными технологиями, такими как искусственный интеллект (*AI*) и большие данные (*Big Data*), что позволяет перейти от простого мониторинга к сложной аналитике, автономному принятию решений и созданию «умных» и адаптивных систем [8].

Интернет вещей прошел длительный путь эволюции – от пророческих идей Николы Теслы и первых экспериментов с сетевыми устройствами до формирования глобальной технологической платформы, трансформирующей все аспекты современного общества. Концепция *IoT*, определяемая как сетевая инфраструктура физических объектов, способных к автономному взаимодействию, обрела свою материальную форму благодаря конвергенции технологий микроэлектроники, беспроводной связи и облачных вычислений.

Количественный критерий, предложенный *Cisco IBSG*, позволил зафиксировать исторический момент (2008-2009 гг.), когда Интернет вещей стал статистически значимым явлением, превзойдя по числу подключенных узлов «Интернет людей». Катализатором этого перехода стало массовое распространение смартфонов, кото-

рые выступили в роли универсальных шлюзов и пользовательских интерфейсов для *IoT*-экосистем.

На текущем этапе *IoT* находится в фазе зрелости и интенсивного роста, что подтверждается многомиллиардными инвестициями и экспоненциальным увеличением количества подключенных устройств. Дальнейшее развитие концепции связано с преодолением ряда вызовов, включая проблемы информационной безопасности, стандартизации, энергоэффективности и защиты приватности, а также с углубленной интеграцией с технологиями искусственного интеллекта, что откроет путь к созданию полностью автономных киберфизических систем.

Таким образом, Интернет вещей является не просто технологическим трендом, а фундаментальным сдвигом в парадигме взаимодействия между человеком, машиной и окружающей средой, определяющим облик Четвертой промышленной революции (Индустрии 4.0).

3. ПРОМЫШЛЕННЫЙ ИНТЕРНЕТ ВЕЩЕЙ (IIoT) КАК ОСНОВА ЦИФРОВОЙ ТРАНСФОРМАЦИИ ПРОМЫШЛЕННОСТИ

3.1. Сущность и отличительные признаки *IIoT*

Промышленный Интернет Вещей – это система объединенных компьютерных сетей и подключенных промышленных объектов со встроенными датчиками и ПО для сбора и обмена данными, с возможностью удаленного контроля и управления в автоматизированном режиме, без участия человека [8].

Ключевые отличия *IIoT* от потребительского *IoT* заключаются в следующих аспектах:

– Требования к надежности и отказоустойчивости: Промышленные системы требуют бесперебойной работы в экстремальных условиях, с минимальным временем простоя.

– Безопасность данных и систем: Последствия кибератак на объекты критической инфраструктуры (энергетика, водоснабжение, транспорт) могут быть катастрофическими.

– Масштабируемость и интеграция: *IIoT*-системы должны интегрироваться с уже существующим оборудованием и корпоративными системами планирования (*ERP, MES*).

– Детерминизм сетей: Во многих промышленных сценариях необходима гарантированно низкая задержка передачи данных, что обеспечивается технологиями типа *TSN* (*Time-Sensitive Networking*).

Таким образом, *IIoT* представляет собой не просто «интернет для заводов», а качественно новую архитектуру производственных и операционных технологий (OT), конвергирующую с информационными технологиями.

3.2. Экономический императив внедрения *IIoT*

Внедрение *IIoT* оказывает значительное влияние на экономику. Это влияние реализуется по трем основным направлениям:

1. На уровне предприятия: Повышение операционной эффективности, сокращение затрат на энергоресурсы и обслуживание, увеличение коэффициента использования оборудования, снижение брака.

2. На уровне отрасли: Оптимизация цепочек поставок, создание прозрачных и гибких логистических систем, формирование отраслевых цифровых платформ.

3. На макроэкономическом уровне: Рост валового национального продукта за счет повышения совокупной производительности труда, стимулирование инноваций и создание новых высокотехнологичных рабочих мест.

Фокус инвестиций на решении конкретных задач, таких как сокращение затрат или повышение эффективности поставок, указывает на прагматичный, ROI-ориентированный подход к внедрению *IIoT*, что является признаком зрелости технологии.

4. КЛЮЧЕВЫЕ ДОМЕНЫ ВНЕДРЕНИЯ И ПРАКТИЧЕСКАЯ ЦЕННОСТЬ IIoT

4.1. Дискретное и непрерывное производство

– Предиктивное техническое обслуживание: Датчики вибрации, температуры и акустики на оборудовании позволяют прогнозировать отказы до их наступления, планировать ремонты в непиковое время и избегать дорогостоящих простоев. Это одна из самых востребованных и экономически эффективных практик *IIoT*.

– Цифровой двойник: Создание виртуальной копии физического актива, процесса или системы. Цифровой двойник позволяет проводить симуляции, оптимизировать параметры работы и анализировать поведение объекта в реальном времени, не вмешиваясь в реальный производственный процесс.

– Контроль качества в реальном времени: Компьютерное зрение и датчики позволяют автоматически выявлять дефекты продукции на ранних стадиях производства, минимизируя потери.

4.2. Коммунальное хозяйство

– Умные сети: *IIoT* позволяет проводить балансировку нагрузки в электросетях, интегрирует возобновляемые источники энергии, автоматически локализует и устраняет аварии. Умные счетчики позволяют вести поквартирный и поэтапный учет потребления ресурсов.

– Управление водоснабжением: Мониторинг качества воды, обнаружение утечек в трубопроводах и оптимизация давления в сети.

4.3. Логистика и розничная торговля

– Управление цепочками поставок: Датчики *GPS* и *RFID* предоставляют данные о местоположении и состоянии грузов (температура, влажность, удары) в режиме реального времени, обеспечивая полную прозрачность и позволяя оперативно реагировать на нарушения.

– Оптимизация складских операций: Автономные роботы-погрузчики, системы динамического складирования и управления запасами на основе актуальных данных.

5. IIoT КАК ТЕХНОЛОГИЧЕСКОЕ ЯДРО ИНДУСТРИИ 4.0

5.1. Эволюция технологических укладов

Промышленный интернет вещей тесно связан с понятием четвертой промышленной революции, или Индустрии 4.0. Чтобы понять его системообразующую роль, необходимо рассматривать его не изолированно, а в контексте конвергенции с другими сквозными технологиями.

Каждая промышленная революция меняла парадигму производства:

– Индустрия 3.0: Автоматизация единичных процессов и роботизация на основе программируемых логических контроллеров.

– Индустрия 4.0: Создание киберфизических систем, где физические объекты (станки, продукция, складские системы) через *IIoT* обмениваются данными с виртуальными моделями, а решения принимаются на основе анализа больших данных и алгоритмов искусственного интеллекта.

5.2. Синергия *IIoT* с другими технологиями Индустрии 4.0

Практическая ценность *IIoT* многократно усиливается при интеграции с другими технологиями из списка Индустрии 4.0:

– *IIoT* + Большие данные и *AI*: IIoT является главным источником больших данных в промышленности. Алгоритмы машинного обучения, применяемые к этим данным, позволяют перейти от мониторинга к предиктивной и прескриптивной аналитике, то есть не только прогнозировать события, но и рекомендовать оптимальные действия [10].

– *IIoT* + Облачные вычисления и Периферийные вычисления: Облака предоставляют масштабируемые ресурсы для хранения и сложной аналитики. Периферийные вычисления позволяют обрабатывать критические данные непосредственно рядом с оборудованием, обеспечивая минимальную задержку и снижая нагрузку на сеть.

– *IIoT* + Автономные роботы: *IIoT* выступает системой координации и управления для флотов автономных роботов, обеспечивая их синхронную работу и взаимодействие с другими элементами производства.

– *IIoT* + *3D*-печать: В гибких производственных ячейках *IIoT*-датчики могут мониторировать состояние *3D*-принтера, а система

на основе этих данных может автоматически заказывать печать необходимой запасной части или оснастки.

Таким образом, *IIoT* является системой фабрики Индустрии 4.0, собирающей сенсорные данные и обеспечивающей обратную связь для интеллектуального управления.

6. ГЕНЕЗИС НОВЫХ БИЗНЕС-МОДЕЛЕЙ

Наиболее основательная трансформация, вызванная *IIoT*, связана не с оптимизацией существующих процессов, а с созданием принципиально новых бизнес-моделей.

1. Переход от продажи продукта к предоставлению услуги: Производитель оборудования (например, промышленных двигателей или лифтов) перестает просто продавать его, а заключает договор на оказание услуги «по результату». Клиент платит за количество моточасов или подъемов, а производитель, используя данные с *IIoT*-датчиков, гарантирует бесперебойную работу, самостоятельно занимаясь предиктивным обслуживанием. Это меняет всю структуру доходов компании.

2. Создание экосистем и отраслевых платформ: Компании-лидеры на основе своих IIoT-решений создают открытые платформы, к которым подключаются поставщики, партнеры и клиенты. Это формирует новые цифровые рынки и экосистемы, где ценность создается совместно.

3. Цифровые двойники как услуга: Появление специализированных провайдеров, которые предлагают малым и средним предприятиям услуги по созданию и эксплуатации цифровых двойников без необходимости инвестировать в дорогостоящую ИТ-инфраструктуру.

Практическая ценность Интернета Вещей, в особенности его промышленного сегмента, заключается в его способности выступать катализатором глубинной операционной и бизнес-трансформации. Она реализуется через три взаимосвязанных уровня:

1. Операционный уровень: Достижение тактических выгод – повышение производительности, сокращение издержек и улучшение качества за счет решений типа предиктивного обслуживания и мониторинга цепочек поставок.

2. Стратегический уровень: Позиционирование *IIoT* как технологического ядра Индустрии 4.0, синергия с *AI*, *Big Data* и облачными технологиями, что ведет к созданию принципиально новых, гибких и интеллектуальных производственных систем.

3. Бизнес-модельный уровень: Наибольший эффект – переход от традиционной продажи продуктов к сервисным моделям и создание цифровых экосистем, генерирующих устойчивые конкурентные преимущества.

Фокус глобальных инвестиций именно на производственных отраслях подтверждает, что *IIoT* перешел из стадии экспериментальных пилотных проектов в стадию зрелой технологии, внедрение которой диктуется экономическим императивом. Дальнейшее развитие практической ценности *IIoT* будет связано с преодолением барьеров, таких как вопросы стандартизации и кибербезопасности, и углублением интеграции с когнитивными технологиями, что откроет путь к полностью автономным и самооптимизирующимся производствам.

ЛАБОРАТОРНАЯ РАБОТА № 1
ПРОЕКТИРОВАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-ИНТЕРФЕЙСА

Цель работы – приобретение навыков проектирования приложения интернета вещей и интерфейса для отображения состояния системы.

Методические указания

В упрощённом и обобщённом виде любая система интернета вещей состоит из трёх слоёв (рис. 1).

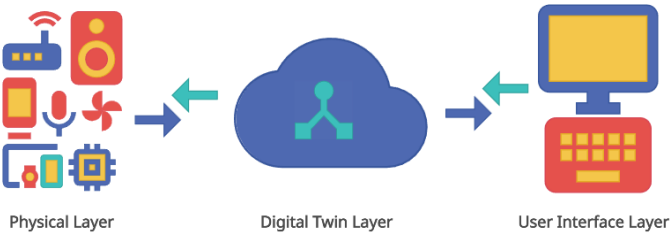

Рис. 1. Обобщённая схема системы интернета вещей

Остановимся подробнее на каждом слое:

Физический слой (*Physical Layer*) – слой реальных вещей, которые взаимодействуют с окружающим миром. На этом слое находятся все устройства, включенные в систему. Работа с этим слоем ограничена возможностями самих устройств, поэтому основная работа с данным слоем заключается в подборе этих самых устройств под ваши требования.

Слой цифровых двойников (*Digital Twin Layer*) – слой, как правило, расположенный на выделенном сервере и представляющий собой набор цифровых двойников реальных устройств. Данный слой отвечает за управление, мониторинг, обработку и обмен информацией между вещами системы. Цифровые двойники позволяют без особых трудностей обрабатывать информацию, поступающую с реальных устройств, поскольку данный слой ограничен лишь вычислительными мощностями сервера.

Слой интерфейсов пользователя (*User Interface Layer*) – слой, предназначенный для отображения информации с устройств и управления ими. Может быть реализован самыми разными способами, например, веб-интерфейс.

Цифровые двойники в системе, как правило представлены набором объектов классов, которые заранее спроектированы. Так, например, можно создать класс для датчика и внести в него все необходимые свойства и методы. Когда нам понадобится включить новый датчик в системы, мы просто можем создать новый объект этого класса.

Таким образом, проектирование слоя цифровых двойников сводится к созданию максимально продуманной диаграммы классов, которая будет включать в себя все необходимые свойства и методы для функционирования системы.

Диаграмма классов *UML* представляет собой набор таблиц различного содержания, которые описывают классы, их атрибуты, методы и отношения между объектами. Основным объектом этой диаграммы, как не сложно догадаться, является Класс. Класс представляет собой шаблон для создания объектов, обеспечивающий начальные значения состояний: инициализацию полей-переменных и реализацию поведения полей и методов. На рис. 2 представлен пример класса.

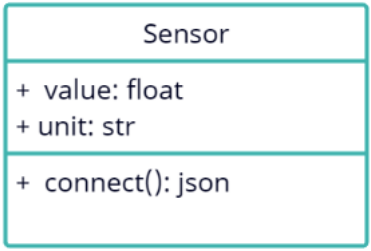

Рис. 2. Класс «Sensor»

Между классами могут (и должны) быть отношения. Отношения бывают разные, но наиболее часто использующиеся из них – это ассоциации и наследование.

Ассоциации показывают связь объектов системы и показываются прямой линией, соединяющей два класса с указанием смыла этого взаимодействия. Также можно указать число объектов, которые могу учувствовать в отношениях, оно записывается в виде интервала или точного числа, которое обозначает количество объектов с той или иной стороны. 1..4 – от одного до четырёх объектов. * – сколько угодно объектов. 1..* – Сколько угодно, но не менее одного объекта. На рис. 3 показан пример ассоциации. Из этого рисун-

ка видно, что множество объектов класса Sensor могут отправлять данные одному объекту класса *MainControlUnit*.

Рис. 3. Пример ассоциации

Наследование отображает отношения между родительским и дочерним классами. Обозначится полой стрелкой, направленной на родительский класс. Классы могут также наследоваться от абстрактного класса. Абстрактный класс – это класс, который лишь описывает некую структуру и набор методов, но не предполагает возможность создания объектов. Такой класс представляет собой некую абстрактную модель, имеющую некоторый объём, но не имеющую содержания. Такой класс может быть полезен при проектировании для создания общего для нескольких классов набора свойств и методов. На Диаграмме такой класс отличается от обычного лишь тем, что его название указано курсивом. На рис. 4 представлен пример наследования от абстрактного класса. В данном случае объект класса Sensor будет иметь не только свойства и методы своего класса, но и также свойства и методы класса Thing.

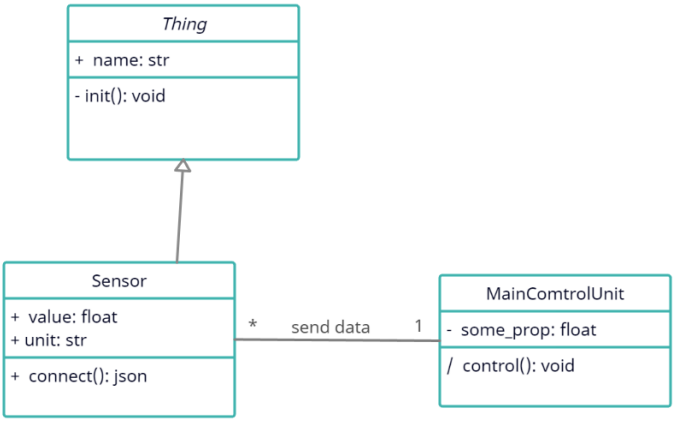

Рис. 4. Наследование от абстрактного класса

Проектирование интерфейса – скорее творческая задача, чем точная наука. Интерфейс может быть реализован в виде схематического рисунка с указанием областей, в которых будет располагаться нужная информация. В рамках данной лабораторной работы будет оцениваться практичность и удобство интерфейса, а не его красота.

Задание

1. В рамках предложенной (или своей) темы спроектируйте систему, состоящую из 4-5 уникальных классов. Подумайте имеют ли классы общие методы и свойства, если таковые есть, выделите их в абстрактный класс. Обозначьте связь элементов системы (Подсказка: не лишним будет создать отдельный объект, который не имеет физического воплощения, а отвечает только за обработку и передачу данных между другими объектами, если это необходимо, в моём примере – это *MainControlUnit*). Подумайте какие данные в системе будет хранить база данных, обозначьте её отдельной сущностью, связями обозначьте какие данные и с каких вещей она должна хранить. Поскольку следующие лабораторные подразумевают написание кода на Python, в котором модификаторы доступа носят рекомендательный характер, ими можно пренебречь.

2. В рамках предложенной (или своей) темы спроектируйте, как минимум два интерфейса. Два интерфейса могут быть нужны, например, для пользователя и администратора. На примере умного дома это может быть интерфейс ребёнка, который только отображает некоторые данные и позволяет управлять некоторыми из вещей и интерфейс родителя, которые может позволять более глубоко настраивать управление домом. Подумайте какие данные лучше скрыть от одного пользователя, какие лучше показать, как лучше их отобразить.

Варианты заданий:

1. Умный дом
2. Умный гараж
3. Умная теплица
4. Умный ресторан
5. Умный склад
6. Умная улица
7. Умная производственная ячейка
8. Умная больничная палата
9. Умное игровое поле (для вашей любимой спортивной игры)
10. Умная система слежения за состоянием человека

Содержание отчёта:

1. Титульный лист
2. Задание
3. Диаграмма классов с описанием всех классов
4. Скриншоты интерфейсов с пояснениями
5. Выводы

Полезные ссылки:

1. https://habr.com/ru/post/511798/
2. https://ux.pub/editorial/10-pravil-proiektirovaniia-intierfieisov-kotoryie-nielzia-narushat-21m6

ЛАБОРАТОРНАЯ РАБОТА № 2
СОЗДАНИЕ ПРИЛОЖЕНИЯ ИНТЕРНЕТА ВЕЩЕЙ И ВЕБ-ИНТЕРФЕЙСА

Цель работы – приобретение навыков создания и запуска приложения интернета вещей.

Методические указания

Для создания приложения интернета вещей необходим набор инструментов, который позволит реализовать веб-приложение, способное взаимодействовать с различным удалённым оборудованием. *Flask* – фреймворк для создания веб-приложений на языке программирования Python. Этот фреймворк обладает необходимым базовым функционалом, а все недостающие детали можно легко восполнить необходимыми модулями.

Для работы с данным фреймворком рекомендуется установить *pycharm professional*, лицензия которой бесплатно предоставляется всем студентам аккредитованных учебных заведений через подписку *GitHub Student Developer Pack*, а также есть пробный период в 30 дней. В данной среде достаточно выбрать тип проекта *flask* и нужные модули будут автоматически установлены. Альтернативный способ установки *Flask* включает ручное создание виртуального окружения и запуск приложения в нем.

После создания проекта мы видим содержание файла *app.py* (рис. 5). Это файл основного приложения, который включает все ссылки, по которым можно переходить. Запустив приложение и вбив в браузере адрес 127.0.0.1:5000 мы увидим созданную приложением веб-страницу (рис. 6).

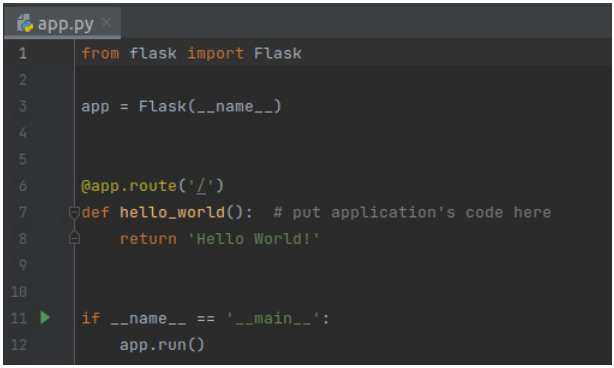

Рис. 5. Содержание файла app.py

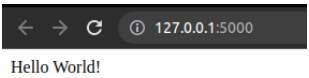

Рис. 6. Автоматически созданная страница

Далее нам необходимо создать в проекте новый файл, который будет содержать наши классы. Для создания класса в языке *Python* используется директива *class*. Затем в скобках указывается класс-родитель, если таковой имеется, по умолчанию *python* использует класс *Base* – базовый класс, являющийся родителем всех прочих.

Внутри каждого класса объявляется метод *__init__*. *__init__* – это системный метод, который будет неявно создан в любом случае, когда класс будет объявлен, однако разработчик может его переопределить, как и любой другой системный метод. Этот метод исполняется в момент создания объекта класса и, как правило, в нём объявляются атрибуты класса. После всех системных методов идут пользовательские методы. Они объявляются, как отдельные функции, за исключение того, что у каждого метода есть входной параметр self, который указывает на текущий объект этого класса. Пример создания класса представлен на рис. 7.

```
class Sensor:
    def __init__(self, unit):
        self.unit = unit
        self.value = 0
        print('sensor has create')

    def connect(self, source):
        return ('connection to ' + source + ' success')
```

Рис. 7. Класс Sensor

Абстрактные классы в *Python* представляются модулем *abc*. Каждый абстрактный класс должен наследоваться от класса *ABC* этого модуля. Как известно, абстрактные классы не могут создавать объекты, а нужны только, чтобы от них наследовались другие классы, которые будут получить их методы и свойства. Перед каждым методом такого класса должен стоять декоратор *«@abstractmethod»*. Однако, когда мы наследуем класс от абстрактного класса, необходимо будет вызвать в методе *__init__* дочернего класса метод __

*init*__ родительского абстрактного класса и добавить в метод вызова __*init*__ дочернего класса все атрибуты, которые были определены в родительском абстрактном классе.

Надо также заметить, что хоть абстрактный класс имеет метод __*init*__, этот метод может быть вызван только в дочернем классе и попытка создать объект абстрактного класса вызовет ошибку. Метод, описанный в абстрактном классе, должен быть также реализован и в классе-потомке.

Ниже на рис. 8 представлен абстрактный класс *Thing* и модифицированный класс *Sensor*.

```
import abc


class Thing(abc.ABC):
    def __init__(self, name):
        self.name = name
        print('create_Thing')

    @abc.abstractmethod
    def connect(self, *args):
        print('Connection start')


class Sensor(Thing):
    def __init__(self, unit, name):
        super().__init__(name)
        self.unit = unit
        self.value = 0
        print('sensor has create')

    def connect(self, source):
        super().connect()
        return ('connection to ' + source + ' success')
```

Рис. 8. класс Sensor, унаследованный от абстрактного класса Thing

Теперь, когда созданы классы, можно приступить к реализации функционала классов. Вернёмся к файлу *app.py*, импортируем наш файл с классами и в функции *hello_world* создадим объект нашего класса, затем обратимся к его атрибуту и вызовем его метод. Ниже представлен код (рис. 9) и скриншоты лога (рис. 10) нашего сервера, а также созданная веб-страница (рис. 11).

```
@app.route('/')
def hello_world():  # put application's code here
    temp_sensor = things.Sensor('C', 'temerature_sensor')
    print(temp_sensor.unit)
    return temp_sensor.connect('temp unit')
```

Рис. 9. Код функции hello_world

```
In folder /home/greed/PycharmProjects/IOTproj
/home/greed/PycharmProjects/IOTproj/venv/bin/python -m flask run
 * Serving Flask app 'app.py'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [30/Sep/2022 19:39:54] "GET / HTTP/1.1" 200 -
create_Thing
sensor has create
C
Connection start
```

Рис. 10. Лог сообщений сервера

Рис. 11. Веб-страница

Задание

Создать приложение на Flask. Реализовать классы и их иерархию, согласно сданной первой лабораторной работе, показать работу методов этих классов. (Не нужно писать предполагаемый метод целиком, на данном этапе достаточно вывести в лог сервера сообщение о том, что метод был запущен)

Содержание отчёта:

6. Титульный лист
7. Задание
8. Код классов и приложения
9. Скриншоты
10. Выводы

Полезные ссылки:

1. https://www.jetbrains.com/ru-ru/pycharm/
2. https://metanit.com/python/tutorial/7.1.php
3. https://docs-python.ru/tutorial/klassy-jazyke-python/abstraktnye-klassy/
4. https://education.github.com/pack
5. https://www.jetbrains.com/ru-ru/pycharm/

ЛАБОРАТОРНАЯ РАБОТА № 3
ОРГАНИЗАЦИЯ ПОЛУЧЕНИЯ И ВЫВОДА ДАННЫХ С УДАЛЕННОГО ОБОРУДОВАНИЯ

Цель работы – приобретение навыков организации получения данных с удаленного оборудования.

Методические указания

Для организации обмена данными с оборудованием обычно использую GET-запросы, в которых с оборудования приходит информация для мониторинга, а от пользователя команды для управления.

В качестве оборудования будем использовать функцию, которая будет автоматически обновлять поля нашего объекта класса. В реальных условиях, между системой и физической вещью необходимо будет выстраивать взаимодействие при помощи веб-запросов, также как и между системой и интерфейсом. На рис. 12 ниже показана функция эмуляции, которая обновляет поле объекта класса, задавая ему случайно значение в заданном диапазоне.

```
def emulate(self):
    self.value = random.randint(15, 25)
```

Рис. 12. Функция эмуляции для вещи Sensor

Эмулятор готов, теперь напишем к нему функцию подключения на стороне сервера. Для этого создадим функцию по аналогии с «*hello_world*», но укажем другой путь роутинга, в моем случае – это «*/connect*». Внутри этой функции вызовем метод connect нашей вещи и передадим его результат в return. Для этого модифицируем метод *connent* таким образом, чтобы он возвращал данные, которые обновляются функцией emulate. Вызовем в методе connect метод emulate. После этого необходимо упаковать данные в *json* и передать, как результат выполнения функции роутинга. Функция роутинга (рис. 13) и модифицированный метод (рис. 14) connect представлены ниже.

```
@app.route('/connect')
def connect():
    return temp_sensor.connect()
```

Рис. 13. Код функции connect

```
def connect(self):
    super().connect()
    self.emulate()
    print('Connection with '+self.name+' success, new value is ' + str(self.value))
    return json.dumps({'value': self.value})
```

Рис. 14. Метод connect вещи Sensor

Для отображения информации, получаемой от эмулятора вещи необходимо разработать веб-страницу с полями для заполнения и *ajax* – запрос для получения этих данных из системы. Ниже представлен пример страницы с полем, в которое записываются данные (рис. 15) и код *ajax* – запроса для получения данных (рис. 16).

Значение | Отправить

Рис. 15. Страница эмулятора вещи Sensor

```
setInterval(get_data,  timeout: 1000)
1 usage
function get_data() {
    $.ajax( url: {
        type: 'GET',
        url: '/connect',
        dataType: 'json',
        contentType: 'application/json',
        data: {},
        success: function (response) {
            document.getElementById( elementId: "value").value = response["value"]
            }
    });
}
```

Рис. 16. Код ajax – запроса для отправки данных

Структура этого запроса следующая:

– *type* – это тип запроса, в нашем случае – GET

– *url* – адрес, на который будет отправлен запрос

– *dataType* – тип данных, который ожидаем от сервера, в нашем случае – *json*

– *contentType*– тип передаваемых данных, в нашем случае – *json*

– *data* –данные запроса, которые будут переданы в систему.

– *success* – это функция, которая будет исполнена в случае, если запрос будет успешно исполнен и получит ответ. В нашем случае ответ – это *json*, в котором передаётся всего один параметр *value*, который передается в элемент с соответствующим названием.

Далее необходимо структурировать наш интерфейс для удобства работы с ним. Запишем наш *js*-код в отдельный файл, добавив к нему функцию *SetInterval*, которая будет вызывать нашу функцию подключения каждую секунду, чтобы подключение выполнялось регулярно, и поместим его в папку *js* в директории *static*, нашу страницу поместим в папку *templates*, добавим ссылки на соответствующие файлы, а также подключим *ajax* для корректной работы *js*-скрипта. В конце добавим стилей в папку *css*, чтобы эмулятор выглядел опрятней. Код страницы эмулятора можно увидеть на рис. 17.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sensor Emulator</title>
    <link rel="stylesheet" type="text/css" href="../static/css/main.css">
    <script src="http://code.jquery.com/jquery-3.3.1.js"></script>
    <script src="../static/js/index.js"></script>
</head>
<body>
    <div id="monitoring_data">
        <input type="text" id="value" placeholder="Значение">
    </div>
</body>
</html>
```

Рис. 17. html-код страницы эмулятора

Далее нам необходимо запустить наш интерфейс на сервере, чтобы он корректно работал с нашей системой. Для этого поменяем return нашей функции «*hello_world*» на return *render_template('sensor_emulator.html')*. Для этого в первую строку файла

```
In folder D:/PyCharm/LabsIOT
D:\PyCharm\LabsIOT\venv\Scripts\python.exe -m flask run
create_Thing
sensor has create
 * Serving Flask app 'app.py'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
127.0.0.1 - - [26/Oct/2023 23:09:43] "GET /connect HTTP/1.1" 200 -
Connection start
Connection with temerature_sensor success, new value is 25
Connection start
Connection with temerature_sensor success, new value is 19
```

Рис. 18. Результат подключения

app.py надо добавить *render_template*. Теперь при заходе по ссылке 127.0.0.1:5000 будет рендерится наш интерфейс.

Пока что наша функция не может обратиться к цифровому двойнику нашей вещи, поскольку он объявлен локально внутри функции *hello_world*, поэтому перенесем объявление в начало скрипта, чтобы объект стал глобальным. Результат работы функции представлен на рис. 18. Можно заметить, что сообщения о создании вещи теперь показываются перед запуском сервера.

Задание

Реализовать эмуляторы вещей для всех классов, созданных в предыдущей лабораторной. Разработать один или несколько интерфейсов, в которые будут передаваться данные от эмуляторов. В рамках данной лабораторной работы необходимо передать в интерфейс только данные для мониторинга. Передача команд, которые осуществляют управление вещью, будет рассмотрена в следующей лабораторной. Для каждой вещи необходимо реализовать собственный ajax-запрос и собственную функцию подключения.

Содержание отчёта:

11. Титульный лист
12. Задание
13. Код классов и приложения
14. Скриншоты
15. Выводы

Код app.py:

```
from flask import Flask, request, render _ template
import things

app = Flask( _ _ name _ _ )
temp _ sensor = things.Sensor('C', 'temerature _ sensor')


@app.route('/connect')
def connect():
    return temp _ sensor.connect()


@app.route('/')
def hello _ world():  # put application's code here
    return render _ template('sensor _ emulator.html')


if _ _ name _ _ == ' _ _ main _ _ ':
    app.run()
```

Код things.py

```
import abc
import json
import random


class Thing(abc.ABC):
    def _ _ init _ _ (self, name):
        self.name = name
        print('create _ Thing')

    @abc.abstractmethod
    def connect(self, *args):
        print('Connection start')


class Sensor(Thing):
    def _ _ init _ _ (self, unit, name):
        super(). _ _ init _ _ (name)
        self.unit = unit
        self.value = 0
        print('sensor has create')

    def connect(self):
        super().connect()
        self.emulate()
        print('Connection with '+self.name+' success, new value is ' +
str(self.value))
        return json.dumps({'value': self.value})

    def emulate(self):
        self.value = random.randint(15, 25)
```

Код index.js

```
setInterval(get _ data, 1000)
function get _ data() {
    $.ajax({
        type: 'GET',
        url: '/connect',
        dataType: 'json',
        contentType: 'application/json',
        data: {},
        success: function (response) {
            document.getElementById("value").value = response["value"]
            }
    });
}
```

Код sensor_emulator.html

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sensor Emulator</title>
    <link rel="stylesheet" type="text/css" href="../static/css/main.
css">
    <script src="http://code.jquery.com/jquery-3.3.1.js"></script>
    <script src="../static/js/index.js"></script>
</head>
<body>
    <div id="monitoring _ data">
        <input type="text" id="value" placeholder="Значение">
    </div>
</body>
</html>
```

ЛАБОРАТОРНАЯ РАБОТА № 4
ОРГАНИЗАЦИЯ ПЕРЕДАЧИ УПРАВЛЯЮЩИХ КОМАНД НА УДАЛЁННОЕ ОБОРУДОВАНИЕ

Цель работы – приобретение навыков организации передачи управляющих команд на удаленное оборудование.

Методические указания

Процесс передачи управляющих команд аналогичен получению данных мониторинга, только управляющие команды передаются, как запрос к оборудованию с командами управления.

В первую очередь модифицируем наш эмулятор таким образом, чтобы в нём появилось поле для ввода отправляемой команды. Вид модифицированного эмулятора представлен на рис. 19.

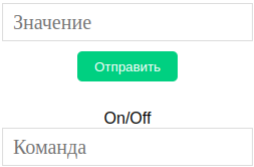

Рис. 19. Эмулятор с полем для отправки данных

Далее необходимо отправить данные на сервер. Сделать это можно при помощи все того же ajax-запроса. Пример представлен на рис. 20.

```
function send_data() {
    $.ajax({
        type: 'GET',
        url: '/connect',
        dataType: 'json',
        contentType: 'application/json',
        data: {
            "value": document.getElementById( elementId: "value").value,
        },
        success: function (response) {

        }

    });
}
```

Рис. 20. Код ajax – запроса для отправки данных

Далее необходимо модифицировать метод connect таким образом, чтобы он обрабатывал *json*-объект, содержащий данные, которые мы хотим передать на оборудование. Внутри этой функции вызовем метод connect нашей вещи и передадим ему наш request. Внутри нашего метода вытащим из нашего *GET*-запроса необходимую информацию и присвоим её нашей вещи. request – это объект, хранящий в себе данные *GET* – запроса, который был передан функции, чтобы получить к нему доступ, его также надо добавить в первую строку файла *app.py*. Код функции роутинга и метода вещи представлен на рис. 21 и рис. 22.

```
@app.route('/connect')
def connect():
    temp_sensor.connect(request)
    return {}
```

Рис. 21. Код функции connect

```
def connect(self, request):
    super().connect()
    self.value = request.args.get('value', '')
    print('Connection with '+self.name+' success, new value is '+self.value)
```

Рис. 22. Метод connect вещи Sensor

Страница управления, а также результат подключению показаны ниже на рис. 23 и рис. 24.

Рис. 23. Страница управления

```
/home/greed/PycharmProjects/IOTproj/venv/bin/python -m flask run
create_Thing
sensor has create
 * Serving Flask app 'app.py'
 * Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
Connection start
Connection with temerature_sensor success, new value is 120
127.0.0.1 - - [19/Oct/2022 23:54:13] "GET /connect?value=120 HTTP/1.1" 200 -
127.0.0.1 - - [19/Oct/2022 23:54:30] "GET /connect?value=73 HTTP/1.1" 200 -
Connection start
Connection with temerature_sensor success, new value is 73
```

Рис. 24. Результат подключения

Задание

Организовать передачу управляющих команд из интерфейса на оборудование для всех вещей. Передачу управляющих команд можно реализовать как при помощи нескольких запросов, по одному для каждой вещи, так и одним запросом, который будет передавать все команды, а их обработкой будет заниматься отдельная функция на сервере. В рамках данной лабораторной работы необходимо реализовать передачу именно управляющих команд, переделывать запросы из прошлой лабораторной не нужно.

Содержание отчёта:

1. Титульный лист
2. Задание
3. Код классов и приложения
4. Скриншоты
5. Выводы

Код sensor_emulator.html:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Sensor Emulator</title>
    <link rel="stylesheet" type="text/css" href="../static/css/main.
css">
    <script src="http://code.jquery.com/jquery-3.3.1.js"></script>
    <script src="../static/js/index.js"></script>
</head>
<body>
<div id="monitoring _ data">
    <input type="text" id="value" placeholder="Значение">
    <button onclick="send _ data()">Отправить</button>
    <br>
    <labek>On/Off</labek>
    <input type="text" id="command" disabled="disabled"
placeholder="Команда">
</div>

</body>
</html>
```

Код things.py:

```
import abc
import json


class Thing(abc.ABC):
```

```
    def _ _ init _ _ (self, name):
        self.name = name
        print('create _ Thing')

    @abc.abstractmethod
    def connect(self, *args):
        print('Connection start')

class Sensor(Thing):
    def _ _ init _ _ (self, unit, name):
        super(). _ _ init _ _ (name)
        self.unit = unit
        self.value = 0
        self.power = "on"
        print('sensor has create')

    def connect(self, request):
        super().connect()
        self.value = request.args.get('value', '')
        print('Connection with '+self.name+' success, new value is
'+self.value)
        return json.dumps({"power": self.power})
```

Приложение 3 app.py

```
from flask import Flask, request, render _ template
import things

app = Flask( _ _ name _ _ )
temp _ sensor = things.Sensor('C', 'temerature _ sensor')

@app.route('/connect')
def connect():
    return temp _ sensor.connect(request)

@app.route('/')
def hello _ world():  # put application's code here
    return render _ template('sensor _ emulator.html')

if _ _ name _ _ == ' _ _ main _ _ ':
    app.run()
```

Код index.js:

```
function send _ data() {
    $.ajax({
        type: 'GET',
        url: '/connect',
        dataType: 'json',
        contentType: 'application/json',
        data: {
```

```
            "value": document.getElementById("value").value,
        },
        success: function (response) {
            document.getElementById("command").value =
response["power"]
        }     });}
```

ЛАБОРАТОРНАЯ РАБОТА № 5
СОЗДАНИЕ И НАСТРОЙКА СИСТЕМЫ УПРАВЛЕНИЯ ОБОРУДОВАНИЕМ НА БАЗЕ ПЛАТФОРМЫ ИНТЕРНЕТА ВЕЩЕЙ

Цель работы – приобретение навыков ограничения получения данных и приведения их к необходимому формату.

Методические указания

Поскольку, обмен данными между системой и интерфейсами происходит при помощи обмена *JSON*-объектами, мы не можем контролировать формат, в котором данные передаются. Из-за этого нам необходимо проверять соответствуют ли данные требуемому формату на стороне системы.

Наиболее эффективный способ это сделать – при помощи конструкции *try-except*, как показано на рис. 25.

```
def connect(self, request):
    super().connect()
    try:
        float(request.args.get('value', ''))
        self.value = request.args.get('value', '')
        print('Connection with ' + self.name + ' success, new value is ' + self.value)
    except ValueError:
        print('New value has not accept, need int but given ' + str(type(request.args.get('value', '')))
    return json.dumps({"power": self.power, "power2": 'off'})
```

Рис. 25 Модифицированный метод для проверки типа данных

Конструкция работает следующим образом:

Все строки в блоке *try* выполняются по очереди до тех пор, пока не произойдёт ошибка выполнения. В обычной программе это привело бы к досрочному завершению программы, однако в данной конструкции это приведёт к переходу в блок *except* с соответствующей ошибке меткой.

В примере единственная строка, способная вызвать ошибку – это строка с функцией *float*(). Эта функция пытается привести данные к тому типу, который мы ожидаем получить, если ей это не удастся, она вызывает ошибку типа *ValueError*.

Как можно понять из функции, если входящие данные могут быть приведены к типу *float*, то функция успешно завершится, пропустив блок except и запишет новые данные в свойства вещи. Однако, если в систему придут данные, которые не могут быть приведены к необходимому типу, то запись новых данных не будет вы-

полнена, а блок except сообщит о том, что был передан не тот тип данных. Пример работы данной функции показан на рис, 26.

```
Connection start
Connection with temerature_sensor success, new value is 22
127.0.0.1 - - [27/Nov/2022 01:32:53] "GET /connect?value=22 HTTP/1.1" 200 -
Connection start
Connection with temerature_sensor success, new value is -22
127.0.0.1 - - [27/Nov/2022 01:32:56] "GET /connect?value=-22 HTTP/1.1" 200 -
127.0.0.1 - - [27/Nov/2022 01:33:00] "GET /connect?value=-22.23 HTTP/1.1" 200 -
Connection start
Connection with temerature_sensor success, new value is -22.23
127.0.0.1 - - [27/Nov/2022 01:33:04] "GET /connect?value=-22.23aa HTTP/1.1" 200 -
Connection start
New value has not accept, need int, but given <class 'str'>
```

Рис. 26 Лог системы с результатом работы функции

Данный метод подходит для любого типа данных, нужно лишь изменить функцию *float*() на нужную вам. Однако со строками всё немного сложнее. Поскольку к строке может быть приведена любая последовательность символов, необходимо проверять не тип данных, а удовлетворяет ли данная строка вашим требованиям. Сделать это можно либо при помощи регулярных выражений, либо банальной проверкой вхождения строки в словарь. Например, если вы передаёте название месяца, то можно сделать проверку на вхождение полученных данных в словарь вида ['Январь', 'Февраль', ..., 'Декабрь'].

Задание

Модифицировать функции подключения вещей таким образом, чтобы все приходящие данные проверялись. Входящие данные иметь, как минимум два различных формата, включая строку. Должно быть использовано хотя бы одно регулярное выражение.

Содержание отчёта:

1. Титульный лист
2. Задание
3. Код классов и приложения
4. Скриншоты
5. Выводы

Полезные ссылки:

Регулярные выражения: https://habr.com/ru/post/349860/

ЛАБОРАТОРНАЯ РАБОТА № 6
РЕАЛИЗАЦИЯ ПОЛУАВТОМАТИЧЕСКИХ И АВТОМАТИЧЕСКИХ РЕЖИМОВ УПРАВЛЕНИЯ ОБОРУДОВАНИЕМ

Цель работы – приобретение навыков ограничения получения данных и приведения их к необходимому формату.

Методические указания

Одним из основных преимуществ систем на базе интернета вещей является возможность обмена данными между оборудованием. Возможность одного устройства получать информацию с другого позволяет осуществлять автоматическое управление определенными элементами системы на основе заданных заранее параметров.

Класс, который осуществляет мониторинг определенных параметров должен иметь для этого особый метод. Этот метод должен быть вызван после получения новых данных, потому что нас интересует мониторинг именно актуальных свойств, но перед отправкой команды управления, потому что эта функция может изменить команду.

Для примера реализуем класс обогреватель, который будет следить за температурой на датчике и включаться, когда она упадёт ниже заданного порога. На рис. 27 приведен код этого класса.

```
class Heater(Thing):
    def __init__(self, name, switch_on_temperature):
        super().__init__(name)
        self.power = 'Off'
        self.switch_on_temperature = switch_on_temperature

    def connect(self):
        super().connect()
        return {"heater_power": self.power}

    def auto_power(self, temperature):
        self.power = 'On' if temperature < self.switch_on_temperature else 'Off'
```

Рис. 27. Класс Heater

Метод *auto_power* принимает, как входной параметр температуру и меняет своё состояние в зависимости от того выше или ниже полученное значение заданного заранее параметра. Данная функ-

ция должна быть вызвана после того, как новое значение температуры будет присвоено датчику. Для этого необходимо немного модифицировать функцию *connect* и соответствующую ей функцию роутинга. Последняя представлена на рис. 28.

```
@app.route('/connect')
def connect():
    response = temp_sensor.connect(request)
    heater.auto_power(temp_sensor.value)
    return json.dumps(response)
```

Рис. 28. Модифицированная функция роутинга

Теперь каждый раз, когда на датчик температуры приходит новое значение, обогреватель проверяет его и меняет свои характеристики в зависимости от полученных данных. На рис. 29 и рис. 30 представлен результат работы автоматического включения и выключения обогревателя с условием включения при температуре меньше 25.

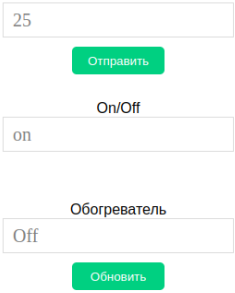

Рис. 29 Обогреватель выключен

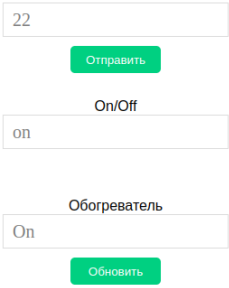

Рис. 30. Обогреватель включен

Задание

Наладить обмен данными между оборудованием внутри системы, создать условия переключения состояния оборудования. Взаимодействие элементов системы должно соответствовать предложенному в первой лабораторной работе проекту.

Содержание отчёта:

1. Титульный лист
2. Задание
3. Код классов и приложения
4. Скриншоты
5. Выводы

Код things.py:

```
import abc


class Thing(abc.ABC):
    def _ _ init _ _ (self, name):
        self.name = name
        print('create _ Thing')

    @abc.abstractmethod
    def connect(self, *args):
        print('Connection start')


class Sensor(Thing):
    def _ _ init _ _ (self, unit, name):
        super(). _ _ init _ _ (name)
        self.unit = unit
        self.value = 0
        self.power = "on"
        print('sensor has create')

    def connect(self, request):
        super().connect()
        try:
            float(request.args.get('value', ''))
            self.value = int(request.args.get('value', ''))
        except ValueError:
            print('New value has not accept, need number but given ' +
str(type(request.args.get('value', '')))
        return {"power": self.power}


class Heater(Thing):
    def _ _ init _ _ (self, name, switch _ on _ temperature):
```

```
        super(). _ _ init _ _ (name)
        self.power = 'Off'
        self.switch _ on _ temperature = switch _ on _ temperature

    def connect(self):
        super().connect()
        return {"heater _ power": self.power}

    def auto _ power(self, temperature):
        self.power = 'On' if temperature < self.switch _ on _ tempera-
ture else 'Off'
```

Код app.py:

```
from flask import Flask, request, render _ template
import things
import json


app = Flask( _ _ name _ _ )
temp _ sensor = things.Sensor('C', 'temerature _ sensor')
heater = things.Heater('Heater1', 25)


@app.route('/connect')
def connect():
    response = temp _ sensor.connect(request)
    heater.auto _ power(temp _ sensor.value)
    return json.dumps(response)


@app.route('/connect _ heater')
def connect _ heater():
    response = heater.connect()
    return json.dumps(response)


@app.route('/')
def hello _ world():  # put application's code here
    return render _ template('sensor _ emulator.html')


if _ _ name _ _ == ' _ _ main _ _ ':
    app.run()
```

ЛАБОРАТОРНАЯ РАБОТА № 7
СОЗДАНИЕ И НАСТРОЙКА СИСТЕМЫ СБОРА ДАННЫХ НА БАЗЕ ПЛАТФОРМЫ ИНТЕРНЕТА ВЕЩЕЙ

Цель работы – приобретение навыков создания системы долгосрочного хранения данных.

Методические указания

Часто возникает необходимость хранить различные данные о работе системы для их дальнейшего просмотра. Это могут быть показания датчиков, лог ошибок, лог успешных операций и много чего ещё. Поскольку задача стоит в сохранении большого количества однотипных данных, разумно будет использовать документо-ориентированную базу данных. Далее будет рассмотрено применение *mongodb* для хранения данных.

Процесс установки *mongodb* и её графического клиента не являются сложной задачей, поэтому подробно описываться не будут. Ссылки на соответствующие инструкции даны в конце файла. Для работы с *mongodb* в Python используется модуль *pymongo*. Его, также, необходимо установить.

Начнём с того, что создадим отдельный класс, который будет отвечать за логирование. Данный класс должен хранить информацию о подключении к базе данных, а также последние записанные данные. Код класса представлен на рис. 31.

```
class Logger:
    def __init__(self, db_name):
        self.current_temperature = []
        self.client = pymongo.MongoClient('mongodb://localhost:27017/')
        self.db = self.client[db_name]

    def insert_temperature(self, new_data):
        if new_data != self.current_temperature:
            self.current_temperature = new_data
            return self.db['Temperature'].insert_one({
                'timeStamp': datetime.datetime.now().strftime("%Y-%m-%d %H:%M:%S"),
                'Temperature': new_data
            })
        print('Value has not changed')
```

Рис. 31. Класс Logger

Ясно видно, что при инициализации объекта класса, создаётся подключение к базе данных, находящейся на том же устройстве,

что и сервер. При этом имя базы данных передаётся, как аргумент. Таким образом, можно создавать несколько экземпляров класса *Logger*, каждый из которых будет подключён к своей базе данных.

Основной функцией логирования является метод *insert_temperature*. Поскольку, нас интересует запись только обновляющихся данных, хранение дублирующийся записей является избыточным. Поэтому, первое, что необходимо сделать – это проверить отличаются ли новые данные от тех, что мы записывали в последний раз. Далее всё, что необходимо сделать – это записать новые данные в созданную для них коллекцию. В данном примере запишется значение текущего момента времени и полученная в этот момент температура.

Всё, что остаётся сделать это создать экземпляр класса и вызвать функцию логирования в том же месте, где мы вызывали функцию, следящую за изменением состояния. Код приведён в приложении. Посмотреть добавляются ли записи в базу данных можно либо в графическом клиенте, либо в консоли, как это показано на рис. 32. Поле _id создаётся *mongodb* автоматически.

```
IOT_logger_db> db.Temperature.find()
[
  {
    _id: ObjectId("639361554c3e16bbdc612216"),
    timeStamp: '2022-12-09 19:24:53',
    Temperature: 23
  },
  {
    _id: ObjectId("639362942e8e9995cabc2050"),
    timeStamp: '2022-12-09 19:30:12',
    Temperature: 2
  },
  {
    _id: ObjectId("639362962e8e9995cabc2051"),
    timeStamp: '2022-12-09 19:30:14',
    Temperature: 3
  },
  {
    _id: ObjectId("639362982e8e9995cabc2052"),
    timeStamp: '2022-12-09 19:30:16',
    Temperature: 12
  }
]
```

Рис. 32 Записи в логе температуры

Задание

Создать систему логирования для долгосрочного хранения данных. Итоговый класс должен содержать, как минимум два различных метода для сохранения данных в разные коллекции.

Содержание отчёта:

1. Титульный лист

2. Задание
3. Код классов и приложения
4. Скриншоты
5. Выводы

Полезные ссылки

1. Установка mongodb: https://www.mongodb.com/docs/manual/administration/install-community/
2. Установка графического клиента: https://www.mongodb.com/docs/manual/tutorial/install-mongodb-on-windows/
3. Онлайн руководство: https://metanit.com/nosql/mongodb/
4. Работа с графическим клиентом: https://metanit.com/nosql/mongodb/1.3.php

Код things.py:

```
import abc
import pymongo
import datetime


class Thing(abc.ABC):
    def _ _ init _ _ (self, name):
        self.name = name
        print('create _ Thing')

    @abc.abstractmethod
    def connect(self, *args):
        print('Connection start')


class Sensor(Thing):
    def _ _ init _ _ (self, unit, name):
        super(). _ _ init _ _ (name)
        self.unit = unit
        self.value = 0
        self.power = "on"
        print('sensor has create')

    def connect(self, request):
        super().connect()
        try:
            float(request.args.get('value', ''))
            self.value = int(request.args.get('value', ''))
        except ValueError:
            print('New value has not accept, need number but given ' +
str(type(request.args.get('value', '')))
        return {"power": self.power}
```

```
class Heater(Thing):
    def _ _ init _ _ (self, name, switch _ on _ temperature):
        super(). _ _ init _ _ (name)
        self.power = 'Off'
        self.switch _ on _ temperature = switch _ on _ temperature

    def connect(self):
        super().connect()
        return {"heater _ power": self.power}

    def auto _ power(self, temperature):
        self.power = 'On' if temperature < self.switch _ on _ tempera-
ture else 'Off'


class Logger:
    def _ _ init _ _ (self, db _ name):
        self.current _ temperature = []
        self.client = pymongo.MongoClient('mongodb://localhost:27017/')
        self.db = self.client[db _ name]

    def insert _ temperature(self, new _ data):
        if new _ data != self.current _ temperature:
            self.current _ temperature = new _ data
            return self.db['Temperature'].insert _ one({
                'timeStamp': datetime.datetime.now().strftime("%Y-%m-%d
%H:%M:%S"),
                'Temperature': new _ data
            })
        print('Value has not changed')
```

Код app.py:

```
from flask import Flask, request, render _ template
import things
import json


app = Flask( _ _ name _ _ )
temp _ sensor = things.Sensor('C', 'temerature _ sensor')
heater = things.Heater('Heater1', 25)
logger = things.Logger('IOT _ logger _ db')


@app.route('/connect')
def connect():
    response = temp _ sensor.connect(request)
    heater.auto _ power(temp _ sensor.value)
```

```
    logger.insert _ temperature(temp _ sensor.value)
    return json.dumps(response)


@app.route('/connect _ heater')
def connect _ heater():
    response = heater.connect()
    return json.dumps(response)


@app.route('/')
def hello _ world():  # put application's code here
    return render _ template('sensor _ emulator.html')


if _ _ name _ _ == '_ _ main _ _ ':
    app.run()
```

ЛАБОРАТОРНАЯ РАБОТА № 8
МОДУЛЬ АНАЛИЗА ДАННЫХ В СИСТЕМАХ ИНТЕРНЕТА ВЕЩЕЙ

Цель работы – приобретение навыков анализа данных в системах интернета вещей.

Методические указания

Для того, чтобы провести анализ данных, для начала необходимо извлечь их из базы данных. Для этого можно использовать тот же модуль, который мы использовали для логирования. На рис. 33 показан метод для извлечения данных температуры из базы данных и поиска её среднего значения.

```
def avg_temperature(self):
    cursor = self.db['Temperature'].find()
    temp_data = []
    for elem in cursor:
        temp_data.append(elem['Temperature'])
    return round(numpy.mean(temp_data), 1)
```

Рис. 33 Метод для вычисления среднего значения температуры

Для начала получим курсор методом *find*. Курсор – это объект, содержащий в себе все записи, возвращённые функцией *find* в виде итерируемой последовательности. Каждая запись представляет собой словарь, где ключи – это названия столбцов. Для того, чтобы получить необходимые нам данные, необходимо пройтись циклом по элементам курсора и из каждого извлечь интересующее нас поле по ключу. После этого применяем функцию расчёта среднего значения массива и вернём её результат.

Добавим вызов описанного метода после функции подключения и добавим его результат в объект, который возвращается этой функцией. Пример показан на рис 34, результат работы на рис. 35.

```
@app.route('/connect')
def connect():
    response = temp_sensor.connect(request)
    heater.auto_power(temp_sensor.value)
    logger.insert_temperature(temp_sensor.value)
    response['avg_t'] = logger.avg_temperature()
    return json.dumps(response)
```

Рис. 34. Вызов функции расчёта среднего значения

Средняя температура

348.7

Рис. 35. Результат работы

Задание

Создать систему анализа данных, хранящихся в базе. Система должна рассчитывать, как минимум две различных характеристики, например среднее и максимальное значение.

Содержание отчёта:

1. Титульный лист
2. Задание
3. Код классов и приложения
4. Скриншоты
5. Выводы

ЛАБОРАТОРНАЯ РАБОТА № 9
НАСТРОЙКА СИСТЕМЫ ВИЗУАЛИЗАЦИИ ДАННЫХ ДЛЯ ЗАДАЧ УПРАВЛЕНИЯ СМАРТ-УСТРОЙСТВАМИ

Цель работы – приобретение навыков визуализации данных в системе интернета вещей.

Методические указания

Визуализация данных, как это можно понять, проводится на стороне интерфейса. Прежде чем приступить к самой визуализации, необходимо передать на интерфейс данные в необходимом формате. Сделать это можно так, как представлено на рис. 36. Метод аналогичен тому, что мы использовали в прошлой лабораторной, за исключением того, что мы накапливаем данные в два массива, а после возвращаем их оба.

```
def temperature_chart(self):
    cursor = self.db['Temperature'].find()
    temp_data = []
    time_data = []
    for elem in cursor:
        temp_data.append(elem['Temperature'])
        time_data.append(elem['timeStamp'])
    return {'temp_data': temp_data, 'time_data': time_data}
```

Рис. 36. Извлечение данных для графика из базы данных

Для визуализации данных используется библиотека *chart.js*. Эту библиотеку необходимо подключить также, как *ajax* строкой:

<script src=»https://unpkg.com/chart.js@3.5.0/dist/chart.min.js»></script>

Далее создадим блок отображения на *html*-странице. Для этого используется тег canvas с классом *chart*. На рис. 37 показан html-код данного блока.

```
<canvas class="chart"></canvas>
<button onclick="create_chart()">Построить график</button>
```

Рис. 37. Блок для отображения графика

Далее приступим к построению самого графика. Функция для получения данных на стороне сервера у нас уже есть, теперь нужно

вызвать её и построить на основе полученных данных график. На рис. 38 показано как это сделать.

```
function create_chart() {
    $.ajax({
        type: 'GET',
        url: '/get_chart',
        dataType: 'json',
        contentType: 'application/json',
        data: {},
        success: function (response) {
            new Chart(
                document.querySelector( selectors: '.chart'),{
                    type: 'line',
                    data: {
                        labels: response['time_data'],
                        datasets: [
                            {
                                label: "Температура",
                                data: response['temp_data'],
                                cubicInterpolationMode: 'monotone'
                            }
                        ]
                    },
                    options: {}
                }
            );
        }
    });
}
```

Рис. 38. Построение графика

Для начала создадим новый объект типа *Chart*. Отображать его будем в соответствующем блоке. Поле *type* обозначает тип графика, *data* – это набор данных, который будет на график отображать. Внутри него поле *labels* – это данные нашей оси *X*, в нашем случае – это массив, содержащий метки времени. Далее идёт блок *datasets* – это данные, которые соответствуют нашим меткам времени. В данном случае там только один объект, но их может быть и несколько. Внутри каждого объекта свой набор полей. label – это подпись

графика, *data* – непосредственно данные для отрисовки графика, *cubicInterpolationMode* нужно для того, чтобы график был закруглённый. Блок *options* нужен для более тонкой настройки графика. На рис. 39 показан результат построения графика.

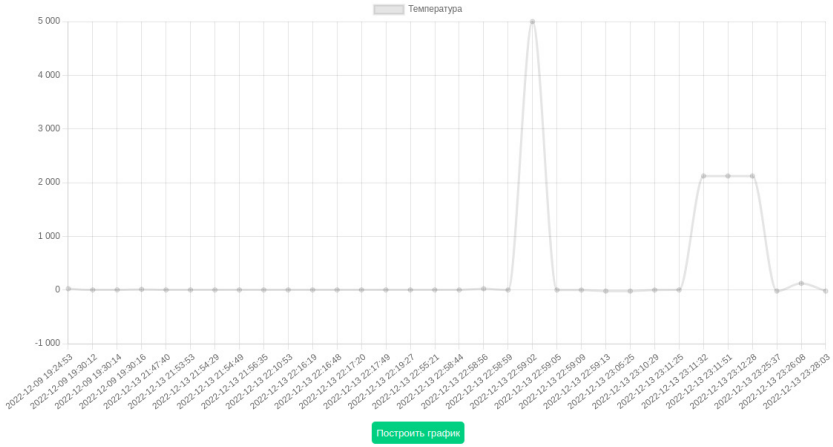

Рис. 39. Построенный график

Задание

Создать систему визуализации данных из базы данных в виде графика. Самостоятельно изучить настройки плагина *chart.js* и сделать график визуально более привлекательным.

Содержание отчёта:

1. Титульный лист
2. Задание
3. Код классов и приложения
4. Скриншоты
5. Выводы

Библиографический список

1. *Ashton, K.* That 'Internet of Things' Thing // RFID Journal. – 2009. – 22 June. – URL: https://www.rfidjournal.com/that-internet-of-things-thing (дата обращения: 15.10.2025).

2. *Убайдов, Ф. Ф.* Интернет вещей (IoT): учеб. пособие / Ф. Ф. Убайдов. – СПб.: Лань, 2022. – 228 с.

3. *Росляков, А. В.* Архитектура и протоколы Интернета вещей / А. В. Росляков, В. А. Петров, А. И. Широков // Транспортная телематика. – 2021. – Т. 11, № 4. – С. 21–30.

4. *Evans, D.* The Internet of Things: How the Next Evolution of the Internet Is Changing Everything // Cisco Internet Business Solutions Group (IBSG) White Paper. – 2011. – April. – P. 1–12.

5. Рекомендация МСЭ-Т Y.4000/Y.2060 (06/2012): Обзор Интернета вещей / Международный союз электросвязи (ITU). – 2015. – 22 с.

6. *Mattern, F.* From the Internet of Computers to the Internet of Things / F. Mattern, C. Floerkemeier // Informatik-Spektrum. – 2010. – Vol. 33, no. 2. – P. 107–121.

7. *Вострикова, А. А.* Мобильные устройства как ключевой элемент экосистемы умного города / А. А. Вострикова, В. В. Синицын // Информационные технологии. – 2023. – Т. 29, № 5. – С. 272–279.

8. *Gilchrist, A.* Industry 4.0: The Industrial Internet of Things / A. Gilchrist. – New York: Apress, 2016. – 258 p.

9. Worldwide Internet of Things Spending Guide [Электронный ресурс] // IDC. – 2023. – Release: V1, 2023. – URL: https://www.idc.com/getdoc.jsp?containerId=IDC_P33198 (дата обращения: 19.10.2025).

10. *Lee, J.* A Cyber-Physical Systems architecture for Industry 4.0-based manufacturing systems / J. Lee, B. Bagheri, H.-A. Kao // Manufacturing Letters. – 2015. – Vol. 3. – P. 18–23.

СОДЕРЖАНИЕ

1. Концептуальные основы Интернета вещей 3
1.1. Сущность и базовое определение.....3
1.2. Ключевые характеристики IoT-систем 3

2. Историческая эволюция концепции Интернета вещей 5
2.1. Ранние предпосылки и пророческие идеи (1920-е – 1980-е гг.) 5
2.2. Технологическая конвергенция и точка перелома (1990-е – 2000-е гг.) 6
2.3. Современный этап и перспективы (2010-е гг. – по настоящее время).....7

3. Промышленный Интернет Вещей (IIoT) как основа цифровой трансформации промышленности.....9
3.1. Сущность и отличительные признаки IIoT.....9
3.2. Экономический императив внедрения IIoT.....9

4. Ключевые домены внедрения и практическая ценность IIoT11
4.1. Дискретное и непрерывное производство 11
4.2. Коммунальное хозяйство 11
4.3. Логистика и розничная торговля 11

5. IIoT как технологическое ядро Индустрии 4.0.....12
5.1. Эволюция технологических укладов 12
5.2. Синергия IIoT с другими технологиями Индустрии 4.012

6. Генезис новых бизнес-моделей 14

Лабораторная работа № 1. Проектирование приложения интернета вещей и веб-интерфейса 16

Лабораторная работа № 2. Создание приложения интернета вещей и веб-интерфейса 21

Лабораторная работа № 3. Организация получения и вывода данных с удаленного оборудования.....25

Лабораторная работа № 4. Организация передачи управляющих команд на удалённое оборудование.....31

Лабораторная работа № 5
Создание и настройка системы управления оборудованием на базе платформы интернета вещей.....36

Лабораторная работа № 6. Реализация полуавтоматических и автоматических режимов управления оборудованием 38

Лабораторная работа № 7. Создание и настройка системы сбора данных на базе платформы интернета вещей 42

Лабораторная работа № 8. Модуль анализа данных в системах интернета вещей 47

Лабораторная работа № 9. Настройка системы визуализации данных для задач управления смарт-устройствами 49

Библиографический список 52

Учебное издание

Д. О. Шевяков,
Д. Д. Савельева

ПРИКЛАДНЫЕ СРЕДСТВА ИНТЕРНЕТА ВЕЩЕЙ

Учебно-методическое пособие

Публикуется в авторской редакции
Компьютерная верстка *А. Н. Колешко*

Подписано к печати 07.02.2025. Формат 60 × 84 1/16.
Усл. печ. л. . Уч.-изд. л. .
Тираж 50 экз. Заказ №.

Редакционно-издательский центр ГУАП
190000, г. Санкт-Петербург, ул. Большая Морская, д. 67, лит. А